implementation, rather than on its logical interface. The good news is that the only easy way to change an object's state in Ruby is by calling one of its methods. Control access to the methods, and you've controlled access to the object. A good rule of thumb is never to expose methods that could leave an object in an invalid state.

Ruby gives you three levels of access control:

- *Public methods* can be called by anyone—no access control is enforced. Methods are public by default (except for initialize, which is always private).
- *Protected methods* can be invoked only by objects of the defining class and its subclasses. Access is kept within the family. We'll talk more about subclasses in Chapter 6, Sharing Functionality: Inheritance, Modules, and Mixins, on page 101.
- *Private methods* cannot be called with an explicit receiver—the receiver is always the current object, also known as self. This means that private methods can be called only in the context of the current object, and with self as the explicit receiver or with the implicit receiver. You can't invoke another object's private methods with normal dot syntax. (There are, however, ways around this using metaprogramming tools that we will discuss in Chapter 22, Metaprogramming, on page 373)

The difference between "protected" and "private" is fairly subtle and is different in Ruby than in most common OO languages. If a method is protected, it may be called by *any* instance of the defining class or its subclasses. If a method is private, it may be called only within the context of the calling object—it is never possible to access another object's private methods directly, even if the object is of the same class as the caller. In practice, it is somewhat rare to see "protected" in use.

Access control in Ruby is determined dynamically, as the program runs, not statically when the program is compiled or interpreted. You will get an access violation only when the code attempts to execute the restricted method.

You specify access levels to methods within class or module definitions using one or more of the three access methods public, protected, and private. You can use each function in three different ways.

If called with no arguments, the three functions set the default access control of subsequently defined methods. This is probably familiar behavior if you're a C# or Java programmer, where you'd use keywords such as public to achieve the same effect. Although this usage looks like a keyword, in Ruby, the access control is actually a method.

```
class MyClass
  # default is "public"
  def method1
    # This method is public
  end

  protected
  # subsequent methods will be "protected"
  def method2
    # This method is protected
  end

  private
  # subsequent methods will be private"
  def method3
```